

Evaluating Different Approaches to Vision-and-Language Navigation: An Experimental Study on a Single GPU

Shubh Jain

<https://github.com/ShubhJain007/evaluating-different-approaches-to-vln>

September 2026

Abstract

This is an evaluation study; it proposes no new method. We take four recent ideas for turning a pretrained vision-language or video model into an instruction-following navigation policy for R2R-CE, build each one on a single 16 GB GPU, and measure its *raw* capability. Each trained model gets one round of offline imitation learning on expert demonstrations and is then evaluated zero-shot in unseen buildings. There is no data flywheel, no RL fine-tuning and no on-policy data. The four ideas are: (i) latent reasoning, via a from-scratch reimplement of LatentPilot, whose *Pilot Token* predicts the visual embedding two steps ahead and is fed back as the only memory; (ii) waypoint *pointing* in the image with a bearing controller, inspired by Mistral AI’s Robostrat Navigate; (iii) the action stream of the Cosmos3-Edge video world model, used zero-shot as a diffusion policy; and (iv) the reasoning tower of the same model, fine-tuned to choose the next command from its own recent video.

Stage 1 of LatentPilot fails the method’s own gate at this scale. The learned Pilot Token predicts future frames $2.25\times$ better than an identity control, yet reaches the goal less often (oracle success 10.7% vs. 17.3%, $n=150$). A held-out test shows why: the same model is 84% accurate with the true next frame in its Pilot slot (its training input) and 31% with its own latent (its test-time input), below the 66% of a model with no slot. Training teaches a shortcut through the privileged frame. Stage 2 (scheduled sampling) recovers some navigation but not the shortcut, even when run three times longer.

Pointing reaches a success rate of **23.3%** (own STOP within 3 m; $n=150$; STOP threshold chosen on `val_seen`), the best in the study, although it came within 3 m in 36.7% of episodes. The zero-shot diffusion policy recovers ego-motion from video but largely ignores the instruction (oracle success 7.5%). The fine-tuned reasoner reaches the goal region about as often (37.5%) but stops there in only **3.3%** of episodes (three strict runs, $n=40$).

Across all four approaches, stopping rather than reaching is the bottleneck. Offline probes ranked candidate models incorrectly twice, and silent implementation errors changed results as much as any modelling choice. They include a success metric that did not require STOP, which we correct here. Code, logs, figures and rollout videos are released.

1 Introduction

Vision-and-Language Navigation (VLN) asks an embodied agent to follow a natural-language route instruction through an unseen building using only egocentric observations [4]. The continuous-environment variant R2R-CE removes the navigation graph. The agent issues low-level actions (move forward 0.25 m, turn 15° , stop) in the Habitat simulator [28, 41] on Matterport3D scans [10].

Recent systems build VLN policies on large pretrained video-language models [13, 51, 52]. Two lines of work propose to change *what* such a model outputs. LatentPilot [20] lets the model reason in latent space. A single *Pilot Token* is trained to predict the next-but-one visual embedding and is carried to the next step as the model’s only memory. The appeal is efficiency and native visual reasoning: one latent per step instead of history frames or chain-of-thought text, which means fewer tokens and faster inference. It reports 51.7–54.0% success on R2R-CE with a 7B model after several rounds of a data flywheel, in which the policy collects trajectories, an expert corrects them and the model is retrained. Its code was not released.

Robostral Navigate [8] from Mistral AI instead asks the model to *point* at the next waypoint in the image, turning navigation into visual grounding. It reaches 73.4% with supervised training of an 8B model on 2.4M simulated trajectories, and 77.4% after online reinforcement learning. In parallel, video world models such as Cosmos [34, 35] ship an action stream and a physical-reasoning tower that might transfer to navigation.

This report is an *evaluation*, not a new method. It asks: *on a single 16 GB GPU, how much of each idea’s promise survives on its own, and why do the approaches fail where they fail?* To measure raw capability we give every trained model exactly one round of offline imitation learning on expert demonstrations. We then evaluate it zero-shot in buildings never seen in training, without LatentPilot’s flywheel, Robostral’s RL fine-tuning, or any on-policy data. We compare four architectures that differ in *what the model is asked to output* (Figure 1):

1. **latent reasoning (LatentPilot)**: discrete action tokens plus a recurrent latent trained to predict future frames;
2. **pointing**: an image-space waypoint and a STOP probability, converted to actions by a geometric controller;
3. **Cosmos3-Edge action model**: ego-pose trajectories sampled from a pretrained video diffusion model, zero-shot;
4. **Cosmos3-Edge reasoner**: a natural-language command from a fine-tuned video reasoner.

Our contributions are:

- A from-scratch reimplementation of LatentPilot, and a controlled test of its Stage 1 claim. With a learned future predictor versus an identity control and everything else fixed, it fails at 2B parameters and 1,665 episodes. A held-out test that changes only the slot content measures a train-time shortcut through the teacher-forced Pilot slot; Stage 2, even three times longer, does not remove it (Section 6.1).
- A supervised-only pointing policy that reaches 23.3% success with a threshold chosen off the test split, with ablations over data scale, history and layer fusion. A layer-wise probe shows that the STOP decision is decodable mid-network (AUC 0.72) but not at the output layer (0.44) (Section 6.2).
- The first, to our knowledge, closed-loop evaluation of Cosmos3-Edge’s action stream and reasoner as VLN policies, including a guidance analysis and the finding that offline next-action accuracy does not predict closed-loop success (Sections 6.3 and 6.4).
- A catalogue of silent implementation errors, each measured, that change results without raising an error (Section 4), and three reported success metrics that separate reaching the goal, ending near it, and stopping at it (Section 3). For every approach we analyse what failed and the most plausible reason.

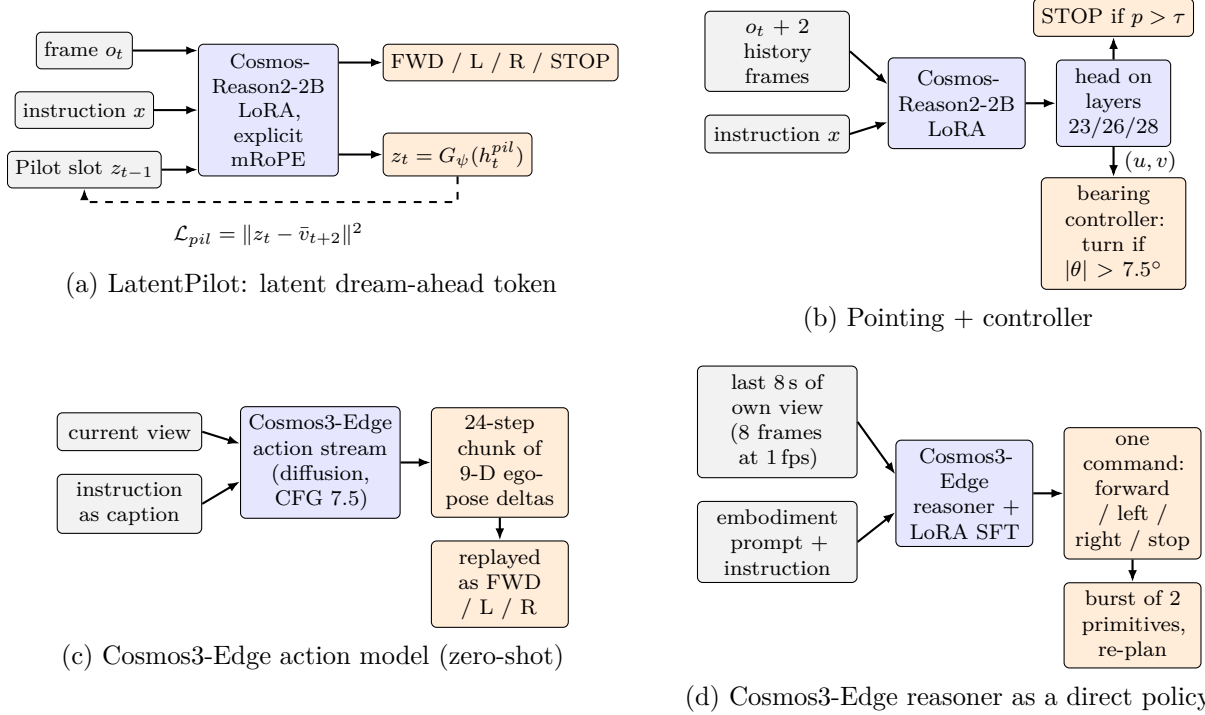


Figure 1: The four architectures. All share the task interface: 448×448 RGB, an English instruction, and the primitives FORWARD 0.25 m, LEFT/RIGHT 15° and STOP, with at most 100 steps per episode.

2 Related work

VLN in discrete and continuous environments. R2R [4] introduced instruction following on the Matterport3D navigation graph. Subsequent work added data augmentation [17], recurrent cross-modal state [22] and explicit history [11]. VLN-CE [28, 29] moved the task into continuous Habitat environments [41, 43]. There, waypoint predictors [23] and topological planners [2] close part of the gap to the graph setting. We follow the standard metrics (NE, OS, SR, SPL [3] and nDTW [25]).

Navigation with large vision-language models. Vision-language-action models transfer web-scale pretraining to control [9, 27]. For navigation, NaVid [51] and Uni-NaVid [52] fine-tune video VLMs to emit actions from egocentric video, and NaVILA [13] separates language-level commands from low-level locomotion. Our reasoner policy (Section 5.4) is closest to NaVILA’s split between command and execution. Our backbones come from the Qwen-VL family [5, 46] with SigLIP encoders [44, 50], through Cosmos-Reason [33, 36]. LatentPilot uses LLaVA-Video [53].

Latent imagination and future prediction. Learning to predict the future in latent space is a long-standing route to better control [18, 19], and feature prediction is also an effective video pretext task [6]. Chain-of-Visual-Thought [38] and LatentPilot [20] insert continuous visual tokens into a VLM’s reasoning. LatentPilot trains them to anticipate future observations.

Pointing and spatial affordances. Grounding-trained VLMs can output image coordinates [15], and pointing has been used as an action representation for manipulation [49]. Robostral Navi-

gate [8], from Mistral AI, trains an 8B VLM to point at the next navigation waypoint, executes it with a learned controller, and refines the policy with online RL. We use the same supervision, but with a geometric controller and no RL.

Video world models and diffusion policies. Cosmos [35] provides video world foundation models; Cosmos3-Edge [34] adds an action stream that denoises ego-motion or robot actions conditioned on video and text. Diffusion policies [14] and flow matching [30] are standard for continuous control, and classifier-free guidance [21] controls how strongly the conditioning text steers sampling.

Training dynamics. Our LatentPilot failure concerns balancing an auxiliary loss against the task loss, the problem addressed by GradNorm [12] and uncertainty weighting [26]. The train/test mismatch of teacher-forced recurrent inputs is the exposure-bias problem [7, 39, 40]. Linear probes [1] guide our layer choice for pointing.

3 Task, data and evaluation protocol

Episodes and scenes. We use R2R-CE [28]. All closed-loop results are on `val_unseen` (1,839 episodes in 11 buildings never seen in training). Training data comes from a reference-path expert: it follows the annotated reference path to within 3m of the goal, rather than the shortest path, which ignores the language. Validated on the full splits, the expert reaches SR 1.000, nDTW 0.753 and NE 2.87m. Three training corpora were used: 6 scans (1,665 episodes, 69,606 steps), 16 scans (4,203 episodes, 171,281 steps) and 61 scans (10,819 episodes, 435,468 steps).

Metrics. NE is the final geodesic distance to the goal. We report three success metrics, from strictest to loosest:

- **SR**, the standard definition: the agent’s own STOP was within 3m of the goal;
- **end-SR**: the episode ended within 3m, whether by STOP or by reaching the 100-step limit;
- **OS**: the agent was within 3m at any point.

Our evaluation script’s “SR” was in fact end-SR. We recover SR from the logged stopping positions where these were recorded, and mark it “n/r” otherwise. SPL weights success by path efficiency, using the geodesic start–goal distance [3]. nDTW [25] is computed after resampling both paths to 0.25m. The literal formula scores a perfect expert at 0.19, and the resampled one at 0.80.

Two evaluation modes. Early in the project the policies did not stop on their own. We therefore built a *diagnostic* mode, in which the episode also ends when the agent first comes within 3m. In this mode SR equals OS by construction, so it measures reaching rather than stopping. The *strict* mode removes that early exit, so the episode ends only at STOP or at the step limit. From it we report both end-SR and, where recoverable, SR.

All LatentPilot and Cosmos3-Edge evaluations, and the original reasoner evaluations, are diagnostic, and we report them as OS. The pointing policy and, in follow-up runs, the reasoner were also evaluated in strict mode.

Evaluation subsets and caveats. Each evaluation takes the first n episodes of `val_unseen`: $n=150$ covers 8 scans and $n=40$ covers 6. Results at different n are therefore not paired. At $n=40$ a 95% binomial interval is about ± 15 points, and at $n=150$ about ± 7.5 . Most STOP thresholds and the turn threshold were selected on `val_unseen` itself and are therefore mildly optimistic. For the

Table 1: Silent implementation errors found by measurement before training. None raises an error.

Issue	Measured effect	Fix
Qwen-VL drops multimodal RoPE [42, 46] when called with <code>inputs_embeds</code>	hidden-state relative L_2 0.42–0.50; top-1 next token agrees on 2/5 prompts	rebuild 3-D position ids explicitly (checked bitwise)
bf16 encoder output depends on batch size	7% relative L_2 for the same frame (fp32: 8×10^{-6})	cache targets $\bar{v}$ in fp32
Paper’s literal token layout (Eq. 5)	21.2% zero-shot instruction following (160 trials, chance 25%) vs. 77.5% for image-first with markers	image-first layout
Loss <code>mean</code> over steps	effective λ becomes 0.15 instead of 0.1 at $T=6$	sum within trajectory
Raw cosine as a predictability gate	black vs. white images already have cosine 0.907	judge retrieval vs. chance
Pointing labels: in-place turns, one shared horizon, furthest waypoint	label/expert agreement 0.689 (fixed: 0.972)	see Section 5.2
Launcher passed history stride 1 instead of 8	7 h of training on the wrong configuration	config checked in log

headline pointing result the STOP threshold is instead chosen on `val_seen` (8 training buildings, 159 new episodes; Appendix D).

Compute. One NVIDIA RTX 4070 Ti SUPER (16 GB). Rendering runs in a separate Habitat worker process [41]. Models use PyTorch [37], Transformers [48], PEFT [32] and Diffusers [45]. All fine-tuning uses LoRA [24], optimised with 8-bit AdamW [16, 31] for the Cosmos-Reason2 models and standard AdamW for the reasoner.

4 Shared backbone and silent implementation errors

Approaches (i) and (ii) use `Cosmos-Reason2-2B` [33], a Qwen-VL-family model [46] with a SigLIP-2 vision encoder [44] and hidden size $d=2048$. A 448×448 frame gives $N_v=196$ visual tokens. The vision encoder is frozen, and LoRA (rank 16, $\alpha=32$) is applied to the query/key/value/output projections of all 28 decoder layers (6.4M trainable parameters). Actions are existing vocabulary tokens read through the native LM head, so an untrained model starts at the uniform loss $\ln 4 = 1.386$.

Before any training we measured every assumption the pipeline depended on (Table 1). Each of these errors leaves training and evaluation running without complaint.

5 Methods

5.1 LatentPilot

Following Hao et al. [20], the frozen encoder maps observation o_t to tokens $v_t = E_\phi(o_t)$, and $\bar{v}_t = \text{Pool}(v_t) \in \mathbb{R}^d$ is their mean. At inference the backbone reads $u_t = [v_t; \text{Tok}(x); \text{PILOT}(z_{t-1})]$ (our image-first order). The action is read from an action-query position h_t^{act} , and the new Pilot Token from the Pilot position, $z_t = G_\psi(h_t^{\text{pil}})$, where G_ψ is a single linear layer. During training the

Pilot slot is teacher-forced with the privileged next-frame embedding $\bar{v}_{t+1}$, and the objective is

$$\mathcal{L} = \underbrace{-\sum_t \log p_\theta(a_t | x, o_t, \bar{v}_{t+1})}_{\mathcal{L}_{act}} + \lambda \underbrace{\sum_{t=1}^{T-2} \|G_\psi(h_t^{pil}) - \bar{v}_{t+2}\|_2^2}_{\mathcal{L}_{pil}}, \quad \lambda = 0.1. \quad (1)$$

Stages. **Stage 0** trains $\mathcal{L}_{act}$ alone. **Stage 0'** adds the recurrent Pilot slot without $\mathcal{L}_{pil}$; this reproduces the paper’s “NaN” ablation row, which, according to the paper’s supplement, still carries z_{t-1} . It compares two slot placements: *A*, the Pilot token last and read for the action; and *B*, a separate action-query token, so that $h^{act} \neq h^{pil}$. **Stage 1** adds $\mathcal{L}_{pil}$.

Controls and initialisation. The *identity control* freezes $G_\psi = I$. It is identical in everything else, so it isolates the effect of learning to predict the future. G_ψ is initialised with zero weights and bias $\mathbb{E}[\bar{v}]$. PyTorch’s default initialisation makes $\lambda\mathcal{L}_{pil}$ 168× larger than $\mathcal{L}_{act}$ at step 0; ours brings it to 0.42×. For reference, persistence ($\bar{v}_t \rightarrow \bar{v}_{t+2}$) scores $\mathcal{L}_{pil}=7.82$ and the constant mean scores 18.48.

Training. One offline round on expert demonstrations; the paper’s data flywheel is not run, and Stage 2 only as a follow-up fine-tune on expert data (Section 6.1). 17,400 steps (2 epochs) at batch 8 on the 6-scan corpus. Learning rate 10^{-4} with 100 warm-up steps and cosine decay to 5×10^{-6} .

5.2 Pointing with a bearing controller

Following Robostral Navigate [8], the policy predicts *where to go* instead of *which action*. Unlike Robostral we train with supervised imitation only (no RL) and use a fixed geometric controller. A linear head on the action-query hidden state outputs the image coordinates (u, v) of a waypoint, a unit direction $(\hat{d}_x, \hat{d}_y)$ in the agent frame, a visibility logit and a STOP logit. It is trained with

$$\mathcal{L} = \|(u, v) - (u^*, v^*)\|^2 \cdot \mathbf{1}[\text{vis}] + (1 - \cos \angle(\hat{d}, d^*)) + 0.5 \text{BCE}_w(\text{stop}) + 0.2 \text{BCE}(\text{vis}), \quad (2)$$

where the STOP term is up-weighted by the inverse base rate (2.5% of steps), capped at 8.

A controller turns 15° toward the predicted bearing if it exceeds 7.5° , and otherwise moves forward 0.25 m. STOP fires when $\sigma(\text{stop}) > \tau$.

Labels. Targets are derived from the reference path, with two horizons. A *pointing* waypoint 24 steps ahead is used for (u, v) . A *control* waypoint, the first reference position at least 0.20 m away within 8 steps, is used for the direction. The naive single-horizon labels agreed with the expert’s own action only 68.9% of the time. Skipping in-place turn steps, which produce $\text{atan2}(0, 0)=0$, and decoupling the horizons raised agreement to 97.2%.

Variants. *History*: two earlier frames (8 steps apart) are added to the input. *Layer fusion*: the head reads the concatenated hidden states of layers 23, 26 and 28 instead of only the last layer. These layers were chosen by a linear probe [1] over all 29 hidden states (Section 6.2). Training uses batch 4–8, learning rate 10^{-4} with cosine decay, and the 6-, 16- or 61-scan corpus.

5.3 Cosmos3-Edge action model

Cosmos3-Edge [34] can be run in a *policy mode*. Given a video frame and an action-caption prompt, it denoises a chunk of 9-D ego-pose deltas (translation and a 6-D rotation). We use the autonomous-driving domain, whose prior transferred best to indoor walking (Section 6.3). The model is prompted with the R2R instruction and sampled with 20 steps and classifier-free guidance [21] of 7.5. Each 24-step chunk is scaled by a translation factor of 7.47, fitted on our rendered video, then replayed as FORWARD/LEFT/RIGHT primitives before re-planning. The model is used zero-shot. A flow-matching post-training run [30] of the action stream (13M LoRA parameters) was started but abandoned after its first logged step; no checkpoint was saved.

5.4 Cosmos3-Edge reasoner as a direct policy

The reasoning tower of Cosmos3-Edge receives the agent’s own recent video and the instruction, and answers with one of four commands. The prompt is given in Appendix A. Each command is executed as a burst of two primitives (0.5 m, or 30°), after which the policy re-plans. STOP is executed after two consecutive stop commands. Zero-shot probes showed that the model narrated the scene as a bystander rather than acting as the agent (Section 6.4). We therefore prefix every prompt with an embodiment statement modelled on NVIDIA’s driving prompt, and fine-tune with LoRA on expert decision points:

- **SFT v1:** 43,036 samples, onset-balanced (forward 50%, stop 25%, turns 25%), and a 16-frame context, which the processor reduces to about 1 s.
- **SFT v3:** 45,000 uniformly sampled decision points (forward 66%, stop 5%) and an 8 s context (120 source frames sampled to 8 at 1 fps), with the vision projector also adapted.

Both use one epoch, learning rate 10^{-4} and gradient accumulation 16. Inference uses no chain-of-thought [47], with sampling at top- p 0.8 and temperature 0.7.

6 Results

Table 2 and Figure 2 summarise closed-loop performance. We discuss each approach in turn.

6.1 LatentPilot: a better predictor, a worse navigator

Stage 0 cannot be the paper’s baseline. A memoryless Stage 0 policy reached 78% training accuracy but never predicted STOP. In closed loop it spun in place: 10 of 12 episodes made zero progress, with 44% LEFT and 43% RIGHT actions against 62% FORWARD in training. The paper’s supplement shows that its “NaN” baseline still carries the Pilot slot recurrently, so we built Stage 0’ to match it.

Slot placement: the zero-shot probe picked the wrong design. Untrained, Design A preserved 59.4% instruction-following accuracy and Design B 29.4%. After identical training (Figure 3), A reached OS 0.0% and B 16.7% ($n=150$). Replacing B’s slot content with a constant or the current frame drops it to 0%, while any learned content keeps 18–22% ($n=60$). B’s slot is therefore load-bearing, and A’s failure is architectural.

The Stage 1 gate fails. Table 3 compares the learned G_ψ against the identity control. The learned Pilot Token reaches a $2.25\times$ lower $\mathcal{L}_{pil}$ (3.02 vs. 6.80; final training values), below the

Table 2: Closed-loop results on R2R-CE `val_unseen`. SR: own STOP within 3m (standard). end-SR: ended within 3m, including at the step limit. OS: within 3m at any point. “n/m”: not measured, because the evaluation ended on arrival (Section 3). *SPL uses end-SR as its success criterion. The bold pointing row uses a STOP threshold chosen on `val_seen`; the others use 0.05–0.20 chosen on this split. Reference rows use 7–8B models on the full split with extra training rounds (flywheel or RL) not used here.

Approach	Variant	n	SR $\uparrow$	end-SR $\uparrow$	SPL* $\uparrow$	OS $\uparrow$	nDTW $\uparrow$	NE $\downarrow$
(i) LatentPilot	Stage 1, learned G_ψ	150	n/m	n/m	–	10.7	0.277	8.50
	identity G_ψ (control)	150	n/m	n/m	–	17.3	0.268	8.17
	+ Stage 2, 50% own z	150	n/m	n/m	–	16.7	0.312	7.98
	+ Stage 2, 75% own z	150	n/m	n/m	–	18.0	0.315	7.99
	+ Stage 2, 75%, $3\times$ longer	150	n/m	n/m	–	19.3	0.298	7.86
(ii) Pointing	16 scans, final layer	150	12.7	13.3	13.0	14.0	0.350	7.34
	+ history, 61 scans	150	17.3	18.0	16.8	22.7	0.394	6.55
	+ longer schedule (40k)	150	22.7	24.7	22.3	31.3	0.345	7.10
	+ fusion, τ tuned on test	150	22.7	31.3	25.4	42.7	0.323	7.22
	+ fusion, τ from val. seen	150	23.3	28.7	24.3	36.7	0.363	7.03
(iii) Edge action	zero-shot, CFG 7.5	40	n/m	n/m	–	7.5	0.276	8.81
(iv) Edge reasoner	SFT v1	40	n/m	n/m	–	17.5	0.365	7.41
	v3 1.5k, 3 diag. runs	40	n/m	n/m	–	43.3	0.364	6.36
	v3 1.5k, 3 strict runs	40	3.3	18.3	12.6	37.5	0.175	7.31
Hao et al. [20]	7B, flywheel	1,839	51.7–54.0	–	47.1–48.5	57.0	–	5.3
Bounhar et al. [8]	8B, SFT + RL	1,839	77.4	–	–	–	–	–

Reasoner rows average three runs (stochastic decoding).

persistence baseline of 7.82. It still reaches the goal less often. With $n=30$ the gap appeared as 10% vs. 40%; at $n=150$ it narrows to 10.7% vs. 17.3%, but it does not reverse.

The cause, measured: a train-time shortcut through the Pilot slot. Following the paper (Equation (1)), the Pilot slot is teacher-forced during training with the *true* embedding of the next frame, $\bar{v}_{t+1}$. At test time it holds the model’s own prediction z_{t-1} . The next frame is the consequence of the current action (the view has rotated, or moved forward), so during training the action can be read off the slot by inverse dynamics. Training accuracy already hints at this (Figure 6a): the memoryless Stage 0 reaches 78%, while every model whose slot is fed $\bar{v}_{t+1}$ reaches 98% (78% vs. 92% at matched steps).

To test it directly we replay 4,306 held-out `val_unseen` expert steps (110 episodes, 11 buildings) through each checkpoint. Observations and instruction are teacher-forced, and only the content of the Pilot slot changes (Table 4, Figure 5). The learned Stage 1 model is 84.3% accurate with the true next frame and **30.8%** with its own latent, less than half of the 66.0% reached by a model with no slot at all. With another episode’s next frame in the slot it falls to 27.8%, so it is reading the slot. Stage 0’, which never trained G_ψ , already shows the dependence (82.0% vs. 56.6%), so the leak comes from the teacher-forced input. Learning to predict the future makes the test-time latent *worse*. The identity control ignores its slot (61–63% under every content), which explains why it navigates better. Across training checkpoints, the gap opens between 2k and 4k steps and then stays constant (Figure 5b).

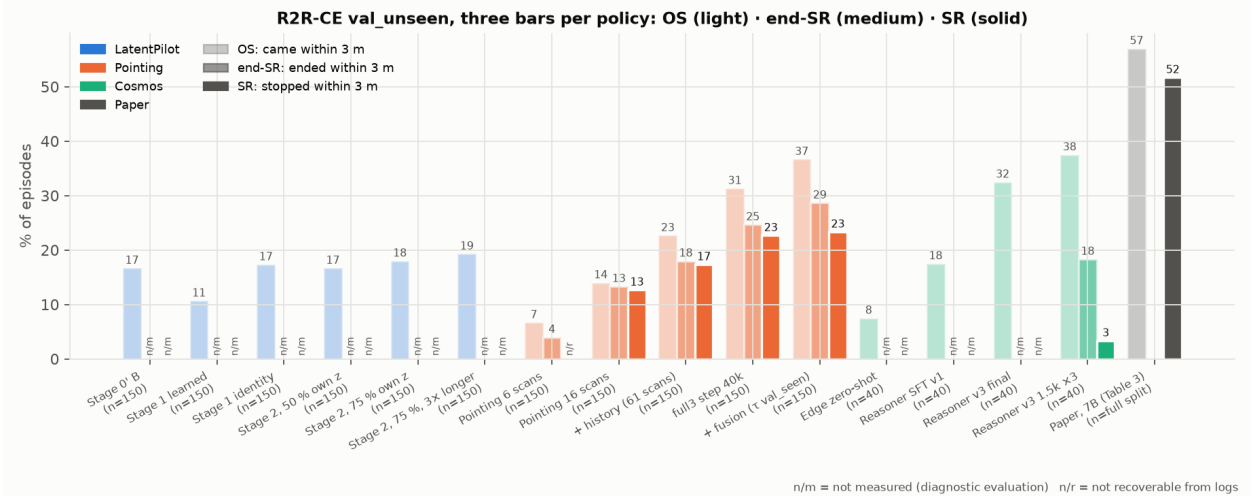


Figure 2: Three bars per policy: OS (light), end-SR (medium) and SR (solid). Only the pointing policy and the reasoner were evaluated without the early exit; the reasoner bars average three strict runs.

Table 3: LatentPilot Stage 0' and Stage 1 (diagnostic; OS in %). Stop = share of episodes ended by the model's own STOP.

Model	n	final $\mathcal{L}_{pil}$	OS	nDTW	NE	stop
Stage 0', Design A (Pilot last)	150	—	0.0	0.244	8.79	39.3%
Stage 0', Design B (action query)	150	—	16.7	0.303	7.95	30.0%
Stage 1, learned G_ψ	30	3.02	10.0	0.306	8.58	0%
Stage 1, identity G_ψ	30	6.80	40.0	0.404	7.40	0%
Stage 1, learned G_ψ	150	3.02	10.7	0.277	8.50	4.7%
Stage 1, identity G_ψ	150	6.80	17.3	0.268	8.17	0%
Stage 1, adaptive λ (step 10k)	150	—	6.7	0.275	8.36	18.0%

Does Stage 2 remove it? Stage 2 scheduled sampling [7] replaces part of the slot inputs with the model's own detached latent. We fine-tune Stage 1 for 2,600 steps ($0.1 \times$ learning rate, same 6 buildings), raising the replaced share to 50% or 75%; the project's rules require approval above 75%. Closed-loop OS improves from 10.7% to 16.7% and 18.0%, the level of the identity control, but the shortcut stays: next-frame accuracy remains 84% and own-latent accuracy only rises to 34%. A three times longer run (7,800 steps at 75%) barely changes this: own-latent accuracy is 35.5%, next-frame accuracy 84.2%, and closed-loop OS 19.3% (within noise of 18.0% at $n=150$), with no STOP issued by the model. More scheduled sampling on expert trajectories does not remove the shortcut. The paper's data flywheel, which trains on the model's own trajectories over several rounds, remains untested at this scale.

A contributing factor: loss-balance drift. With $\lambda = 0.1$ the two terms of Equation (1) are balanced only at initialisation (Figure 4). $\mathcal{L}_{act}$ falls $51 \times$ over training while $\mathcal{L}_{pil}$ falls only $4.8 \times$. The ratio $\lambda \mathcal{L}_{pil} / \mathcal{L}_{act}$ therefore grows from 0.52 to 1.03 at step 4k, 3.2 at step 10k and 5.5 at the end, and most of the gradient ends up serving frame prediction.

An adaptive schedule that holds the ratio at 0.9, in the spirit of GradNorm [12], removed the drift (λ fell from 0.17 to 0.04) but did not recover navigation (OS 6.7% at step 10k), so drift is not

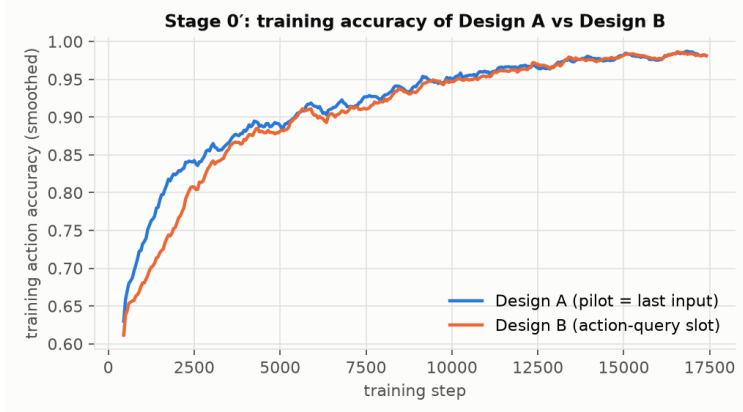


Figure 3: Stage 0' training accuracy for the two Pilot-slot designs. They are nearly identical in training but differ completely in closed loop (OS 0.0% vs. 16.7%).

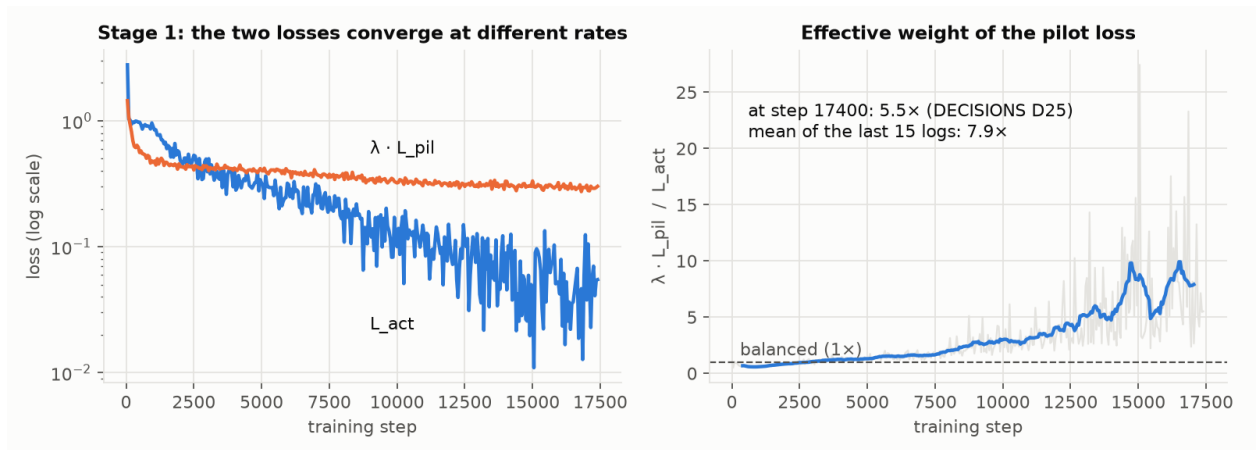


Figure 4: Stage 1 loss balance with $\lambda=0.1$. Left: the two loss terms. Right: their ratio $\lambda \mathcal{L}_{pil} / \mathcal{L}_{act}$, which starts at about 0.5 and ends $5.5\times$ larger.

the main cause. At 2B parameters, 1,665 episodes and without the flywheel, predicting the future did not make a better navigator. This finding is specific to this setting and does not refute the 7B result.

6.2 Pointing: the best success rate, without RL

Data is the binding constraint. This is a supervised-only test of the Robostral idea: no RL fine-tuning and a geometric controller. Trained identically for 20k steps, the 6-scan policy reached end-SR 4.0% although its training loss was lower (0.076 vs. 0.396), while the 16-scan policy reached 13.3% (Table 5). Moving to 61 scans with frame history raised end-SR to 18.0%, a longer schedule to 24.7%, and layer fusion to 31.3%. Re-evaluated with stopping positions logged, SR (own STOP within 3m) is 12.7%, 17.3%, 22.7% and 22.7% respectively. Choosing the STOP threshold on `val_seen` (8 training buildings, 159 new episodes; SR 39.3/35.3/31.3/24.7% at $\tau=0.10/0.15/0.20/0.30$) selects $\tau=0.10$, which gives **SR 23.3%** on `val_unseen`, within a point of the test-tuned 22.7%.

Table 4: Held-out action accuracy (%) with only the Pilot slot changed; 4,306 val_unseen steps.

Model	true next frame	own latent	current frame	other episode	no slot
Stage 0 (memoryless)	—	—	—	—	66.0
Stage 0' (slot, no $\mathcal{L}_{pil}$)	82.0	56.6	44.5	27.8	—
Stage 1, learned G_ψ	84.3	30.8	45.1	27.8	—
Stage 1, identity G_ψ	63.1	61.2	63.2	62.1	—
Stage 2, 50% own z	84.3	33.6	53.0	27.9	—
Stage 2, 75% own z	83.7	33.9	54.5	27.6	—
Stage 2, 75% own z , 3 $\times$ longer	84.2	35.5	55.9	29.6	—

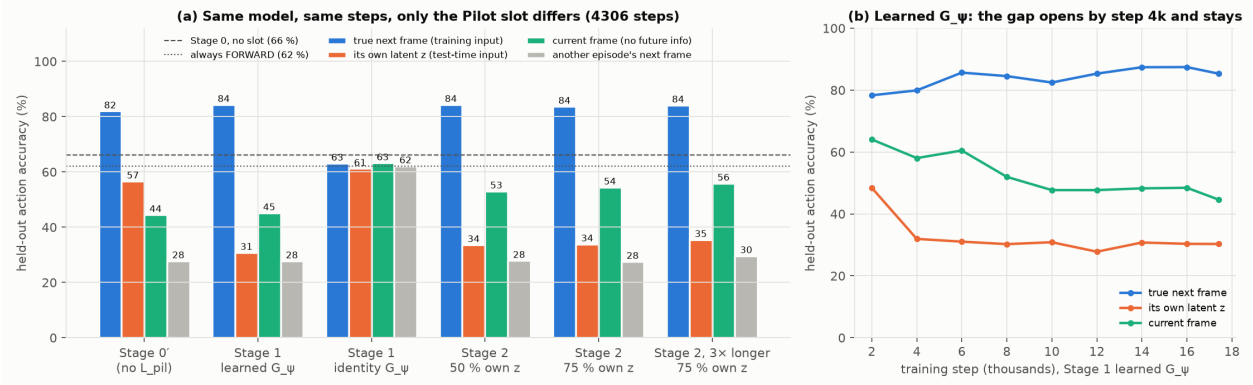


Figure 5: The Pilot-slot shortcut, measured. (a) Held-out action accuracy of each checkpoint with only the slot content changed. (b) The learned Stage 1 model over training: own-latent accuracy collapses by step 4k and stays low while next-frame accuracy stays high.

The policy uses the instruction, and STOP depends on it most. Teacher-forced on 900 held-out steps, action agreement is 0.764 with the true instruction, 0.686 with another episode's, and 0.603 with none. The separation between STOP and non-STOP probabilities collapses from $5.1\times$ to $1.0\times$ once the instruction is swapped.

STOP lives mid-network. A linear probe on each of the 29 hidden states (2,000 training and 1,000 held-out frames) finds STOP at AUC 0.718 in layer 15. At the final layer, which the head was reading, the AUC is 0.441, below chance (Figure 7, left). A head reading layers 23, 26 and 28 reached SR 22.7% (end-SR 31.3%), the same SR as the final-layer head at 40k.

A matched comparison at equal training shows no benefit, however (end-SR 22.7% vs. 22.0%, SR 12.7% vs. 16.0%; Table 5). Results over training are also non-monotonic (Figure 8). The size of the fusion effect is therefore not established.

What fails, and why. Of the 150 episodes, 22.7% stop within 3m, 8.7% run out of steps within 3m, 23.3% stop elsewhere, and 45.3% run out of steps elsewhere (Figure 9a). Only 49% of the policy's own stops are within 3m. There are three causes.

Under-turning. On 2,500 teacher-forced steps, agreement is 0.901 on FORWARD but only 0.55 on turns. Missed turns become FORWARD (LEFT $\rightarrow$ FWD 0.39, RIGHT $\rightarrow$ FWD 0.43) and almost never go the wrong way (LEFT $\rightarrow$ RIGHT 0.06). The cause is regression shrinkage: the predicted bearing is $0.424\times$ the true one (Figure 7, right). Expert turns average 41° but predictions average 27° .

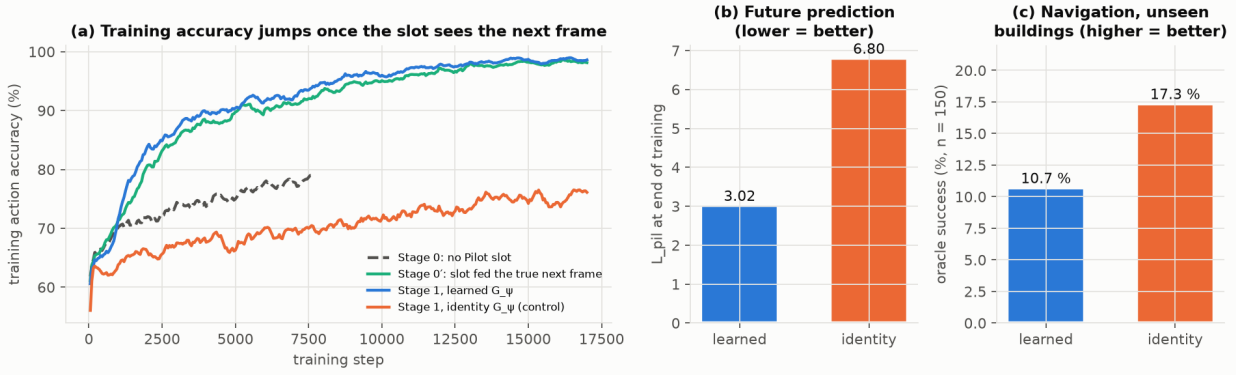


Figure 6: Training-time view of the failure. (a) Training accuracy jumps once the Pilot slot is fed the true next frame. (b) The learned Pilot Token predicts the future better, yet (c) navigates worse in unseen buildings than the identity control.

Table 5: Pointing ablations without the early exit, $n=150$ (8 scans). τ : STOP threshold. SR is recoverable only where stopping positions were logged. The history run used stride 1 because of a launcher error.

Variant	scans	epochs	τ	SR	end-SR	SPL*	OS	NE
final layer, no history	6	2.30	0.05	n/r	4.0	3.7	6.7	8.37
final layer, no history	16	0.93	0.05	12.7	13.3	13.0	14.0	7.34
+ history ($K=2$, stride 1)	61	0.35	0.05	17.3	18.0	16.8	22.7	6.55
+ history ($K=2$, stride 8), step 40k	61	0.37	0.10	22.7	24.7	22.3	31.3	7.10
+ layer fusion 23/26/28, step 120k	61	1.10	0.20	22.7	31.3	25.4	42.7	7.22
<i>matched comparison at $\tau=0.20$:</i>								
final layer, step 60k (0.55 ep.)	61	0.55	0.20	16.0	22.0	18.9	41.3	7.99
layer fusion, step 80k (0.73 ep.)	61	0.73	0.20	12.7	22.7	18.7	41.3	7.90

Lowering the turn threshold does not help in closed loop (diagnostic OS of 35.3, 41.3, 42.0, 42.7 and 40.0% at thresholds of 3, 4, 5, 7.5 and 10°), because a lower threshold produces zig-zagging. *Weak STOP*. STOP is 2.5% of the labels, and the signal is linearly decodable mid-network but not at the output layer. The policy passes close to the goal without stopping, or stops at a similar-looking place (Figure 13b,c). *No recovery*. Imitation learning only ever sees expert states, so the policy never learns to recover from its own drift. This is exactly the capability Robostrat’s online RL phase adds (+4 points on top of a far stronger supervised model), and we did not run it.

6.3 Cosmos3-Edge action model: motion prior without language

Zero-shot inverse dynamics transfers. From our rendered walking video, the `av` (driving) domain recovers per-frame yaw with correlation 0.837 and turn direction 96.4% (384 frames, 11 scans). The `camera_pose` domain reaches only 0.288. On video re-rendered at a fixed 15 fps, yaw correlation reaches 0.987, and the translation scale fits $s=7.47$ (forward correlation 0.92).

Instructions barely steer it. With short commands (“turn left”/“turn right”) the sign of the turn is correct 82/86/95/95% of the time at guidance 1/3/5/7.5 (Figure 10). With full R2R instructions, turn direction agrees with the expert 59.4% of the time at guidance 1 (53.1% with a

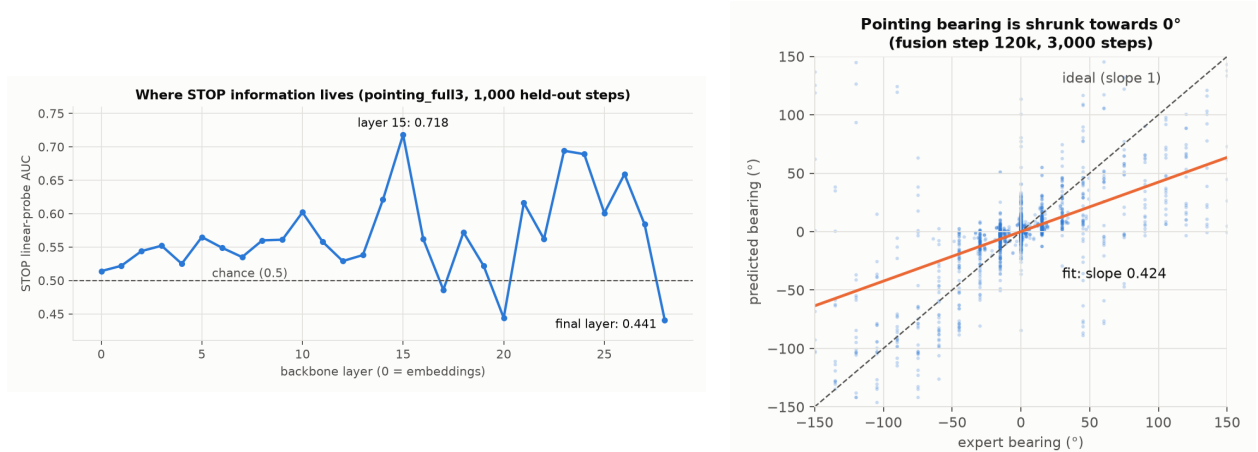


Figure 7: Left: STOP linear-probe AUC per layer; the output layer is below chance. Right: the predicted bearing is $0.42\times$ the expert’s, so many turns fall under the 7.5° threshold.

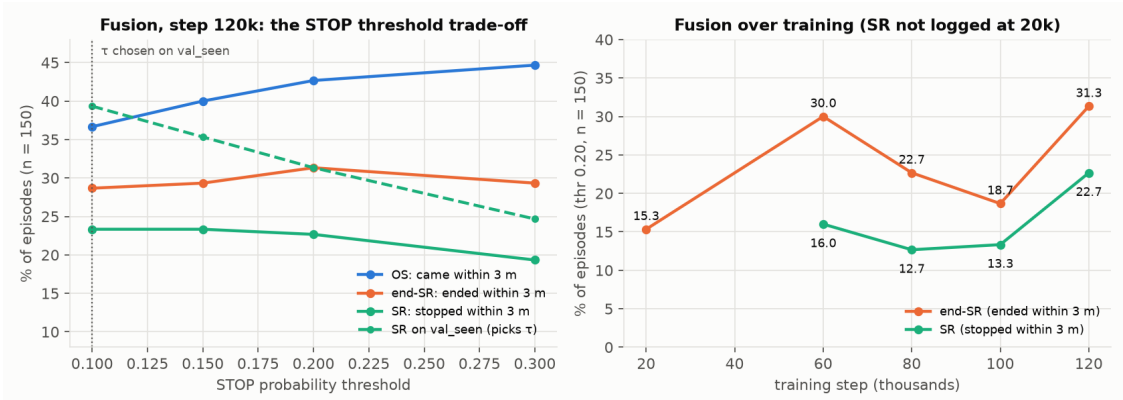


Figure 8: Pointing with layer fusion. Left: raising the STOP threshold τ trades premature stops for overshoots. OS keeps rising, end-SR peaks at 0.20, and SR is flat up to 0.20 and then falls. Right: end-SR and SR over training at $\tau=0.20$ (stopping positions were not logged at 20k).

swapped instruction), and 67.5% vs. 49.5% at guidance 7.5.

What fails, and why. In closed loop ($n=40$) the policy reached OS 7.5%, and 75% of episodes ended early. There are two causes. First, the instruction is out of distribution: the action stream was pretrained on short motion or scene captions, not multi-sentence routes, and a fixed prompt, “The camera moves forward.”, did at least as well (15.0%, $n=20$). Second, it stalls: many sampled chunks contain too little motion to become even one primitive, and two such chunks in a row end the episode. The motion prior is real, but the language has to come from elsewhere.

6.4 Cosmos3-Edge reasoner: reaching without stopping

Zero-shot, it does not know it is the agent. Given a still frame, the reasoner produces plausible-looking waypoint lists (Figure 11). Asked for the next action, however, it narrates the scene as a bystander. For the first frame in Figure 11 it answered “A person enters the room through the archway, drawn by the reflection in the mirror or the painting’s subject.” With the

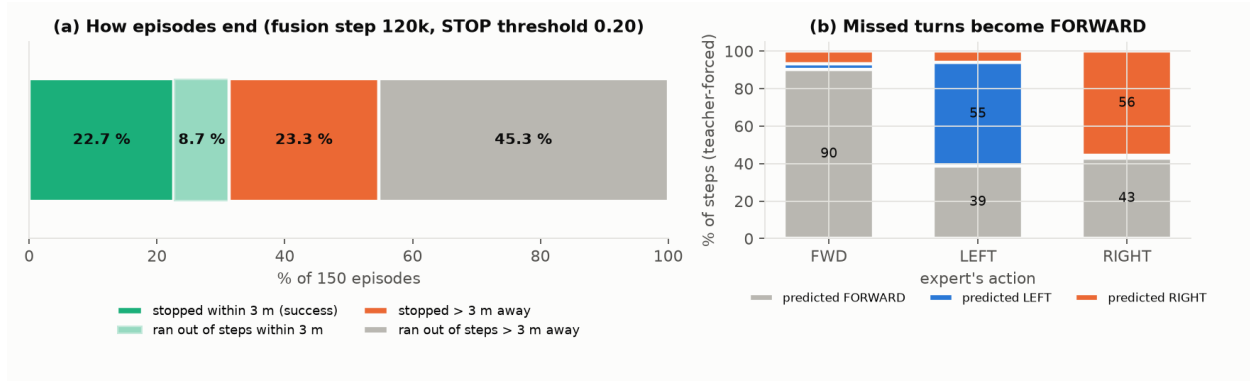


Figure 9: Pointing with layer fusion. (a) How the 150 episodes end. (b) Teacher-forced predictions by expert action: missed turns become FORWARD.

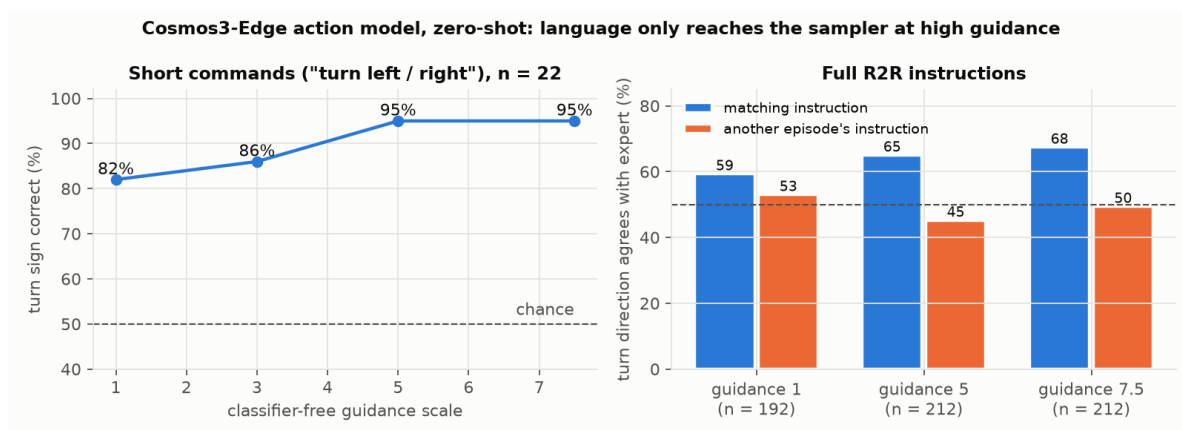


Figure 10: Cosmos3-Edge action model, zero-shot. Language influences the sampled motion only at high classifier-free guidance, and full instructions steer it weakly.

embodiment prompt, zero-shot next-action accuracy on 160 decision points is 32.5% (33.1% with thinking), below the 54.4% majority baseline.

Fine-tuning, and a probe that misleads. Table 6 compares probe accuracy with closed-loop success. SFT v1 has the best probe accuracy, 67.5%, but in closed loop it over-turns and over-stops: STOP ends 65% of episodes, and its step-1,250 checkpoint reaches OS 0%. SFT v3 has lower probe accuracy (63.1%, with STOP recall only 0.10) yet reaches OS 32.5% at the final checkpoint and 47.5% at step 1,500 in its first diagnostic run. Two repeat runs gave 47.5% and 35.0% (mean 43.3%; decoding is stochastic). Evaluated strictly in three runs, it reaches 3m in 37.5% of episodes but stops there in only **2.5, 2.5 and 5.0%** (mean 3.3%). Because v3 changed three factors at once (sampling, context length, projector), we cannot attribute its gain to any one of them.

What fails, and why. Each version reproduces its training label mix (Figure 12). v1 was trained on 25% STOP labels and ends 65% of its episodes by stopping, always more than 3m from the goal. v3 was trained on 5% STOP, rarely stops, and its episodes end by reaching the goal region or running out of steps. Deciding to stop also requires knowing how much of the instruction has been completed, which a window of at most 8s of video without longer memory cannot provide.

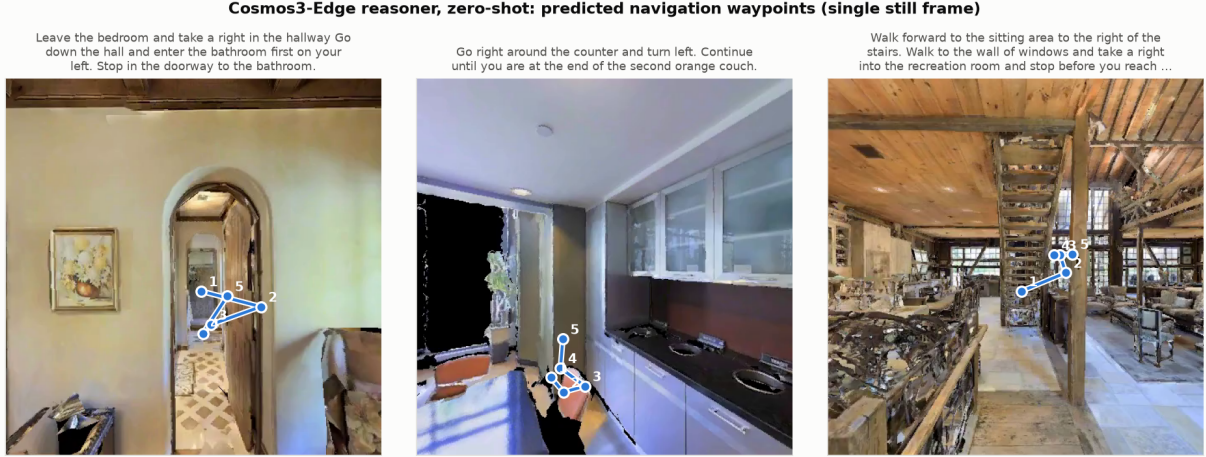


Figure 11: Zero-shot reasoner: predicted navigation waypoints on single frames (coordinates normalised to 0–1000).

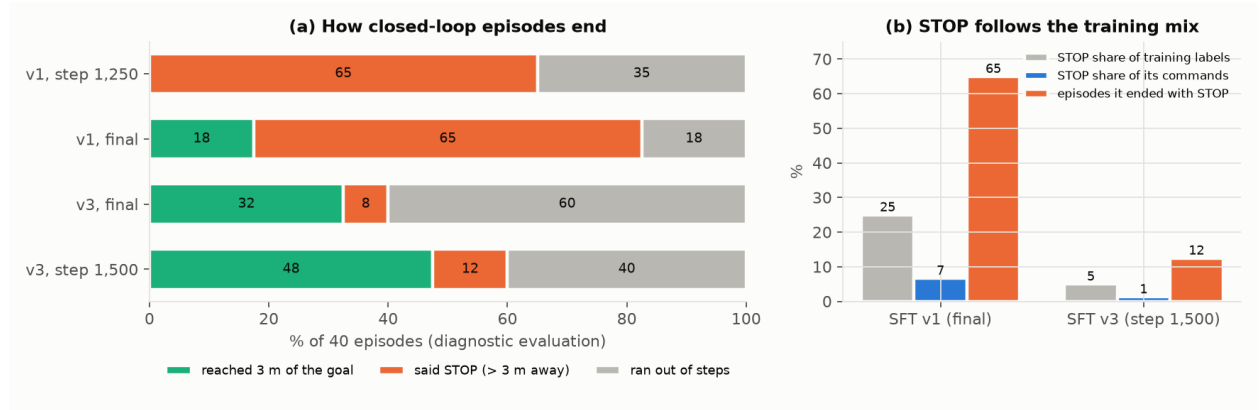


Figure 12: Reasoner. (a) How the 40 closed-loop episodes end (diagnostic evaluation). (b) STOP behaviour follows the STOP share of the training labels.

Training only on expert frames means the model never practises recovering from its own errors.

7 Discussion

Stopping is the bottleneck. Every approach reached the goal region more often than it stopped there. For the pointing policy the gap between OS and SR is 13 points at the threshold chosen on `val_seen`, and the reasoner stopped inside 3 m in 3.3% of episodes while reaching 3 m in 37.5%. The two sides of that failure point to different fixes. The STOP signal exists in the backbone (layer-15 AUC 0.72), and it depends strongly on the instruction. A dedicated STOP head is the one structural change that let a policy stop at all. We expect the next gains to come from supervising STOP on progress through the instruction rather than on proximity alone.

Evaluate the claim, not a proxy. A diagnostic evaluation that ends on arrival made SR identical to OS for most of this project. Our first public write-up reported one of these numbers

Table 6: Reasoner: offline next-action probe (160 decision points, majority baseline 54.4%) vs. closed-loop OS ($n=40$, diagnostic).

Model	context	probe acc.	closed-loop OS	nDTW	own STOP
zero-shot (embodiment prompt)	16 frames	32.5	–	–	–
SFT v1, step 1,250	16 frames	65.6	0.0	0.280	65%
SFT v1, final	16 frames	67.5	17.5	0.365	65%
SFT v3, final	8 s	63.1	32.5	0.304	7.5%
SFT v3, step 1,500	8 s	–	47.5	0.389	12.5%

as SR. Our own “strict” SR also counted episodes that merely ended near the goal at the step limit, which inflated the best pointing result from 22.7% to 31.3%. Both are corrected here. Offline probes also misled twice: a zero-shot probe preferred the wrong Pilot-slot design, and the reasoner with the best next-action accuracy navigated worst. Teacher-forced accuracy measures neither compounding error nor recovery, which is the exposure-bias argument [7, 40] in miniature.

Data and representation over architecture. The largest single improvements for the pointing policy came from more buildings (4.0→13.3%) and from correct labels (0.69→0.97 agreement). Frame history and layer fusion helped less clearly. The LatentPilot result adds a caution: an auxiliary objective trained with privileged inputs can raise training accuracy through a shortcut, and only closed-loop evaluation reveals it.

Raw capability versus full recipes. Every number here comes from one offline round of imitation learning. The parts of each method we did not run are the parts aimed at our observed failures: LatentPilot’s flywheel and scheduled sampling target the train/test slot mismatch, and Robostral’s RL targets recovery. The gap between our results and the published ones should be read in that light.

Pretrained video priors. Cosmos3-Edge’s action stream carries a usable ego-motion prior but not instruction grounding. Its reasoner, once told it is the agent and fine-tuned, is the strongest “reacher”. Combining the two, with the reasoner deciding and the diffusion model executing, was implemented but did not finish on our hardware (5 of 18 completed episodes reached the goal region).

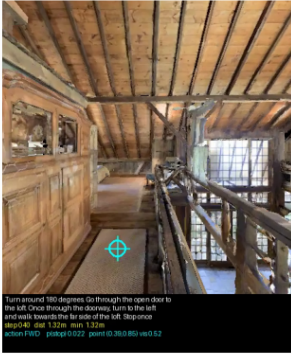
8 Limitations

Evaluations are small ($n=40$ –150) and unpaired across approaches, and they cover 6–8 of the 11 unseen buildings. The headline pointing threshold was chosen on `val_seen`; other thresholds were tuned on the evaluation split. Every method was trained for one offline round only (no flywheel, RL or DAgger). Reasoner rows average three stochastic runs, which varied by up to 12 points. Several runs were stopped before finishing their schedules: the long pointing runs completed 1.1–1.2 of 1.5–3 planned epochs. Stage 2 was run only on expert data (up to 7,800 steps, at most 75% own latents); a 16-scan Stage 1, the data flywheel, and most arms of a prompt/policy ablation for the Cosmos models were never run. Our backbone (2B) and data (6–61 buildings) differ from LatentPilot’s (7B, full data), so our results describe this scale only.

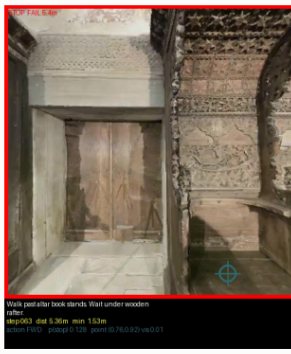
(a) LatentPilot, same episode: expert | Stage 1 step 10k (reaches 2.8 m) | Stage 1 final (never within 9 m)



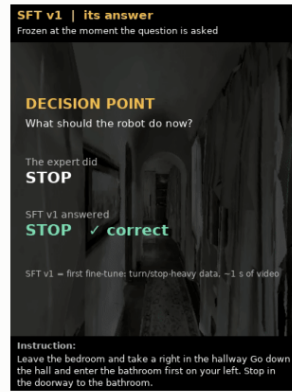
(b) Pointing: 1.3 m from the goal, $p(\text{stop}) = 0.02$; it never stops



(c) Pointing: stops 5.4 m short (it passed within 1.5 m)



(d) Reasoner SFT v1, decision probe at the goal: stop ✓



(e) Reasoner SFT v3, same moment: move forward ✗



Figure 13: Rollouts in unseen buildings. (a) LatentPilot: same episode for the expert, Stage 1 at 10k steps and the final model. (b, c) Pointing failure modes. (d, e) Reasoner decision probes, not rollouts: the same moment of an expert’s walk, answered by SFT v1 (d) and SFT v3 (e). Videos are in the repository.

9 Conclusion

On a single consumer GPU, with one offline training round and zero-shot evaluation, supervising *where to go* with a pointing head and a geometric controller gave the best success rate (23.3%). A fine-tuned video reasoner reached the goal region about as often but stopped there in 3.3% of episodes. LatentPilot’s Stage 1, because of a measured train-time shortcut through its Pilot slot that Stage 2 does not remove, and a zero-shot video diffusion policy did not produce competitive navigators at this scale. In every case the limiting factor was the decision to stop, together with the gap between what an offline metric measures and what the closed loop requires.

Reproducibility. Code, all training and evaluation logs, per-experiment notes, the scripts that regenerate every figure from the logs, and rollout videos are in the repository,

<https://github.com/ShubhJain007/evaluating-different-approaches-to-vln>

Matterport3D-derived media are for non-commercial academic use under the Matterport3D terms of use.

Acknowledgements. Parts of the implementation, experiment orchestration and the drafting of this report were done with an AI coding assistant (Claude, Anthropic). Every number reported here is taken from the logs in the repository.

References

- [1] Guillaume Alain and Yoshua Bengio. Understanding intermediate layers using linear classifier probes, 2018. URL <https://arxiv.org/abs/1610.01644>.
- [2] Dong An, Hanqing Wang, Wenguan Wang, Zun Wang, Yan Huang, Keji He, and Liang Wang. ETPNav: Evolving Topological Planning for Vision-Language Navigation in Continuous Environments, 2024. URL <https://arxiv.org/abs/2304.03047>.
- [3] Peter Anderson, Angel Chang, Devendra Singh Chaplot, Alexey Dosovitskiy, Saurabh Gupta, Vladlen Koltun, Jana Kosecka, Jitendra Malik, Roozbeh Mottaghi, Manolis Savva, and Amir R. Zamir. On Evaluation of Embodied Navigation Agents, 2018. URL <https://arxiv.org/abs/1807.06757>.
- [4] Peter Anderson, Qi Wu, Damien Teney, Jake Bruce, Mark Johnson, Niko Sünderhauf, Ian Reid, Stephen Gould, and Anton van den Hengel. Vision-and-Language Navigation: Interpreting visually-grounded navigation instructions in real environments, 2018. URL <https://arxiv.org/abs/1711.07280>.
- [5] Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibao Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, et al. Qwen2.5-VL Technical Report, 2025. URL <https://arxiv.org/abs/2502.13923>.
- [6] Adrien Bardes, Quentin Garrido, Jean Ponce, Xinlei Chen, Michael Rabbat, Yann LeCun, Mahmoud Assran, and Nicolas Ballas. Revisiting Feature Prediction for Learning Visual Representations from Video, 2024. URL <https://arxiv.org/abs/2404.08471>.
- [7] Samy Bengio, Oriol Vinyals, Navdeep Jaitly, and Noam Shazeer. Scheduled Sampling for Sequence Prediction with Recurrent Neural Networks, 2015. URL <https://arxiv.org/abs/1506.03099>.
- [8] Abdelaziz Bounhar, Abhijeet Somani, Aditi Kabra, Adrian Valente, Adrien Petralia, Adrien Sade, Alan Jeffares, Albert Jiang, Aleksandr Timashov, Alexandre Cahill, et al. Robostral Navigate, 2026. URL <https://arxiv.org/abs/2607.20785>.
- [9] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn, et al. RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control, 2023. URL <https://arxiv.org/abs/2307.15818>.
- [10] Angel Chang, Angela Dai, Thomas Funkhouser, Maciej Halber, Matthias Nießner, Manolis Savva, Shuran Song, Andy Zeng, and Yinda Zhang. Matterport3D: Learning from RGB-D Data in Indoor Environments, 2017. URL <https://arxiv.org/abs/1709.06158>.
- [11] Shizhe Chen, Pierre-Louis Guhur, Cordelia Schmid, and Ivan Laptev. History Aware Multimodal Transformer for Vision-and-Language Navigation, 2023. URL <https://arxiv.org/abs/2110.13309>.

- [12] Zhao Chen, Vijay Badrinarayanan, Chen-Yu Lee, and Andrew Rabinovich. GradNorm: Gradient Normalization for Adaptive Loss Balancing in Deep Multitask Networks, 2018. URL <https://arxiv.org/abs/1711.02257>.
- [13] An-Chieh Cheng, Yandong Ji, Zhaojing Yang, Zaitian Gongye, Xueyan Zou, Jan Kautz, Erdem Biyik, Hongxu Yin, Sifei Liu, and Xiaolong Wang. NaVILA: Legged Robot Vision-Language-Action Model for Navigation, 2025. URL <https://arxiv.org/abs/2412.04453>.
- [14] Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion Policy: Visuomotor Policy Learning via Action Diffusion, 2024. URL <https://arxiv.org/abs/2303.04137>.
- [15] Matt Deitke, Christopher Clark, Sangho Lee, Rohun Tripathi, Yue Yang, Jae Sung Park, Mohammadreza Salehi, Niklas Muennighoff, Kyle Lo, Luca Soldaini, et al. Molmo and PixMo: Open Weights and Open Data for State-of-the-Art Vision-Language Models, 2024. URL <https://arxiv.org/abs/2409.17146>.
- [16] Tim Dettmers, Mike Lewis, Sam Shleifer, and Luke Zettlemoyer. 8-bit Optimizers via Block-wise Quantization, 2022. URL <https://arxiv.org/abs/2110.02861>.
- [17] Daniel Fried, Ronghang Hu, Volkan Cirik, Anna Rohrbach, Jacob Andreas, Louis-Philippe Morency, Taylor Berg-Kirkpatrick, Kate Saenko, Dan Klein, and Trevor Darrell. Speaker-Follower Models for Vision-and-Language Navigation, 2018. URL <https://arxiv.org/abs/1806.02724>.
- [18] David Ha and Jürgen Schmidhuber. World Models, 2018. URL <https://arxiv.org/abs/1803.10122>.
- [19] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering Diverse Domains through World Models, 2024. URL <https://arxiv.org/abs/2301.04104>.
- [20] Haihong Hao, Lei Chen, Mingfei Han, Changlin Li, Dong An, Yuqiang Yang, Zhihui Li, and Xiaojun Chang. LatentPilot: Scene-Aware Vision-and-Language Navigation by Dreaming Ahead with Latent Visual Reasoning, 2026. URL <https://arxiv.org/abs/2603.29165>.
- [21] Jonathan Ho and Tim Salimans. Classifier-Free Diffusion Guidance, 2022. URL <https://arxiv.org/abs/2207.12598>.
- [22] Yicong Hong, Qi Wu, Yuankai Qi, Cristian Rodriguez-Opazo, and Stephen Gould. A Recurrent Vision-and-Language BERT for Navigation, 2021. URL <https://arxiv.org/abs/2011.13922>.
- [23] Yicong Hong, Zun Wang, Qi Wu, and Stephen Gould. Bridging the Gap Between Learning in Discrete and Continuous Environments for Vision-and-Language Navigation, 2022. URL <https://arxiv.org/abs/2203.02764>.
- [24] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-Rank Adaptation of Large Language Models, 2021. URL <https://arxiv.org/abs/2106.09685>.
- [25] Gabriel Ilharco, Vihan Jain, Alexander Ku, Eugene Ie, and Jason Baldridge. General Evaluation for Instruction Conditioned Navigation using Dynamic Time Warping, 2019. URL <https://arxiv.org/abs/1907.05446>.

- [26] Alex Kendall, Yarin Gal, and Roberto Cipolla. Multi-Task Learning Using Uncertainty to Weigh Losses for Scene Geometry and Semantics, 2018. URL <https://arxiv.org/abs/1705.07115>.
- [27] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, et al. OpenVLA: An Open-Source Vision-Language-Action Model, 2024. URL <https://arxiv.org/abs/2406.09246>.
- [28] Jacob Krantz, Erik Wijmans, Arjun Majumdar, Dhruv Batra, and Stefan Lee. Beyond the Nav-Graph: Vision-and-Language Navigation in Continuous Environments, 2020. URL <https://arxiv.org/abs/2004.02857>.
- [29] Jacob Krantz et al. VLN-CE: Vision-and-Language Navigation in Continuous Environments (code and R2R-CE data). GitHub repository, 2020. URL <https://github.com/jacobkrantz/VLN-CE>.
- [30] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow Matching for Generative Modeling, 2023. URL <https://arxiv.org/abs/2210.02747>.
- [31] Ilya Loshchilov and Frank Hutter. Decoupled Weight Decay Regularization, 2019. URL <https://arxiv.org/abs/1711.05101>.
- [32] Sourab Mangrulkar, Sylvain Gugger, Lysandre Debut, Younes Belkada, Sayak Paul, Benjamin Bossan, and Marian Tietz. PEFT: State-of-the-art Parameter-Efficient Fine-Tuning methods. GitHub repository, 2022. URL <https://github.com/huggingface/peft>.
- [33] NVIDIA. Cosmos-Reason2-2B. Hugging Face model card, 2025. URL <https://huggingface.co/nvidia/Cosmos-Reason2-2B>.
- [34] NVIDIA. Cosmos3-Edge. Hugging Face model card, 2026. URL <https://huggingface.co/nvidia/Cosmos3-Edge>.
- [35] NVIDIA, Niket Agarwal, Arslan Ali, Maciej Bala, Yogesh Balaji, Erik Barker, Tiffany Cai, Prithvijit Chattopadhyay, Yongxin Chen, et al. Cosmos World Foundation Model Platform for Physical AI, 2025. URL <https://arxiv.org/abs/2501.03575>.
- [36] NVIDIA, Alisson Azzolini, Junjie Bai, Hannah Brandon, Jiaxin Cao, Prithvijit Chattopadhyay, Huayu Chen, Jinju Chu, Yin Cui, et al. Cosmos-Reason1: From Physical Common Sense To Embodied Reasoning, 2025. URL <https://arxiv.org/abs/2503.15558>.
- [37] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library, 2019. URL <https://arxiv.org/abs/1912.01703>.
- [38] Yiming Qin, Bomin Wei, Jiaxin Ge, Konstantinos Kallidromitis, Stephanie Fu, Trevor Darrell, and XuDong Wang. Chain-of-Visual-Thought: Teaching VLMs to See and Think Better with Continuous Visual Tokens, 2026. URL <https://arxiv.org/abs/2511.19418>.
- [39] Marc’Aurelio Ranzato, Sumit Chopra, Michael Auli, and Wojciech Zaremba. Sequence Level Training with Recurrent Neural Networks, 2016. URL <https://arxiv.org/abs/1511.06732>.

- [40] Stephane Ross, Geoffrey J. Gordon, and J. Andrew Bagnell. A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning, 2011. URL <https://arxiv.org/abs/1011.0686>.
- [41] Manolis Savva, Abhishek Kadian, Oleksandr Maksymets, Yili Zhao, Erik Wijmans, Bhavana Jain, Julian Straub, Jia Liu, Vladlen Koltun, Jitendra Malik, Devi Parikh, and Dhruv Batra. Habitat: A Platform for Embodied AI Research, 2019. URL <https://arxiv.org/abs/1904.01201>.
- [42] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. RoFormer: Enhanced Transformer with Rotary Position Embedding, 2023. URL <https://arxiv.org/abs/2104.09864>.
- [43] Andrew Szot, Alex Clegg, Eric Undersander, Erik Wijmans, Yili Zhao, John Turner, Noah Maestre, Mustafa Mukadam, Devendra Chaplot, Oleksandr Maksymets, et al. Habitat 2.0: Training Home Assistants to Rearrange their Habitat, 2022. URL <https://arxiv.org/abs/2106.14405>.
- [44] Michael Tschannen, Alexey Gritsenko, Xiao Wang, Muhammad Ferjad Naeem, Ibrahim Al-abdulmohsin, Nikhil Parthasarathy, Talfan Evans, Lucas Beyer, Ye Xia, Basil Mustafa, et al. SigLIP 2: Multilingual Vision-Language Encoders with Improved Semantic Understanding, Localization, and Dense Features, 2025. URL <https://arxiv.org/abs/2502.14786>.
- [45] Patrick von Platen, Suraj Patil, Anton Lozhkov, Pedro Cuenca, Nathan Lambert, Kashif Rasul, Mishig Davaadorj, Dhruv Nair, Sayak Paul, William Berman, et al. Diffusers: State-of-the-art diffusion models. GitHub repository, 2022. URL <https://github.com/huggingface/diffusers>.
- [46] Peng Wang, Shuai Bai, Sinan Tan, Shijie Wang, Zhihao Fan, Jinze Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, et al. Qwen2-VL: Enhancing Vision-Language Model’s Perception of the World at Any Resolution, 2024. URL <https://arxiv.org/abs/2409.12191>.
- [47] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models, 2023. URL <https://arxiv.org/abs/2201.11903>.
- [48] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, et al. HuggingFace’s Transformers: State-of-the-art Natural Language Processing, 2020. URL <https://arxiv.org/abs/1910.03771>.
- [49] Wentao Yuan, Jiafei Duan, Valts Blukis, Wilbert Pumacay, Ranjay Krishna, Adithyavairavan Murali, Arsalan Mousavian, and Dieter Fox. RoboPoint: A Vision-Language Model for Spatial Affordance Prediction for Robotics, 2024. URL <https://arxiv.org/abs/2406.10721>.
- [50] Xiaohua Zhai, Basil Mustafa, Alexander Kolesnikov, and Lucas Beyer. Sigmoid Loss for Language Image Pre-Training, 2023. URL <https://arxiv.org/abs/2303.15343>.
- [51] Jiazhao Zhang, Kunyu Wang, Rongtao Xu, Gengze Zhou, Yicong Hong, Xiaomeng Fang, Qi Wu, Zhizheng Zhang, and He Wang. NaVid: Video-based VLM Plans the Next Step for Vision-and-Language Navigation, 2024. URL <https://arxiv.org/abs/2402.15852>.

- [52] Jiazhao Zhang, Kunyu Wang, Shaoan Wang, Minghan Li, Haoran Liu, Songlin Wei, Zhongyuan Wang, Zhizheng Zhang, and He Wang. Uni-NaVid: A Video-based Vision-Language-Action Model for Unifying Embodied Navigation Tasks, 2025. URL <https://arxiv.org/abs/2412.06224>.
- [53] Yuanhan Zhang, Jinming Wu, Wei Li, Bo Li, Zejun Ma, Ziwei Liu, and Chunyuan Li. LLaVA-Video: Video Instruction Tuning With Synthetic Data, 2025. URL <https://arxiv.org/abs/2410.02713>.

A Reasoner prompt

The user turn is the concatenation of the following (verbatim from `src/nav/cosmos_prompts.py`). The agent’s recent video is attached as a video input.

You are a mobile robot navigating inside a building. The video depicts the observation from the robot’s own forward-facing camera at eye level, ending at the present moment.

This is the overall task that the agent is trying to complete: "{instruction}"

What should be the next action of the agent?

Choose the robot’s next action. Answer with exactly one of: move forward, turn left, turn right, stop.

B Training configurations

	LatentPilot St. 1	Pointing (fusion)	Edge action	Reasoner SFT v3
Base model	Cosmos-Reason2-2B	Cosmos-Reason2-2B	Cosmos3-Edge	Cosmos3-Edge (reasoner)
Trainable	LoRA r16 + G_ψ	LoRA r16 + head (49k)	none (zero-shot)	LoRA r16 + projector (6.9M)
Data	6 scans, 69.6k steps	61 scans, 435k steps	–	45k decision points
Schedule	17.4k steps $\times$ 8	124k of 163k steps $\times$ 4	–	1 epoch, accum. 16
LR	10^{-4} cosine	10^{-4} cosine	–	10^{-4}
Inference	argmax action	$\tau=0.20$, turn 7.5°	CFG 7.5, 20 steps, chunk 24	top- p 0.8, T 0.7, 2-step bursts

C Full strict evaluation grid for the fusion policy

Table 7: Every evaluation of the layer-fusion pointing policy without the early exit ($n=150$, turn threshold 7.5°). SR: own STOP within 3 m (n/r where stopping positions were not logged); end-SR: ended within 3 m.

Step	τ	SR	end-SR	SPL*	OS	nDTW	NE
20k	0.05	n/r	4.0	4.0	6.0	0.328	7.90
20k	0.10	n/r	8.7	7.3	10.0	0.315	7.92
20k	0.20	n/r	15.3	11.7	19.3	0.286	7.77
60k	0.20	16.0	30.0	24.3	48.0	0.278	7.13
60k	0.35	4.0	24.7	17.3	50.0	0.196	7.50
60k	0.50	0.7	23.3	15.2	50.0	0.178	7.60
80k	0.15	12.0	20.7	17.0	37.3	0.269	7.89
80k	0.20	12.7	22.7	18.7	41.3	0.248	7.90
80k	0.30	4.7	19.3	14.1	45.3	0.178	8.32
100k	0.15	16.0	20.0	18.1	37.3	0.295	8.27
100k	0.20	13.3	18.7	16.1	40.0	0.258	8.45
100k	0.30	8.0	16.7	13.4	44.0	0.189	8.85
120k	0.10	23.3	28.7	24.3	36.7	0.363	7.03
120k	0.15	23.3	29.3	24.4	40.0	0.351	7.05
120k	0.20	22.7	31.3	25.4	42.7	0.323	7.22
120k	0.30	19.3	29.3	23.6	44.7	0.287	7.59

D STOP threshold chosen on val_seen

Table 8: Layer-fusion policy (step 120k) on val_seen: 8 training buildings, 159 episodes not used in training (strict). The threshold with the best SR, $\tau=0.10$, is then applied to val_unseen (Table 2).

τ	SR	end-SR	OS	NE
0.10	39.3	41.3	50.0	5.80
0.15	35.3	40.0	54.7	5.96
0.20	31.3	38.0	56.0	6.13
0.30	24.7	33.3	59.3	6.49